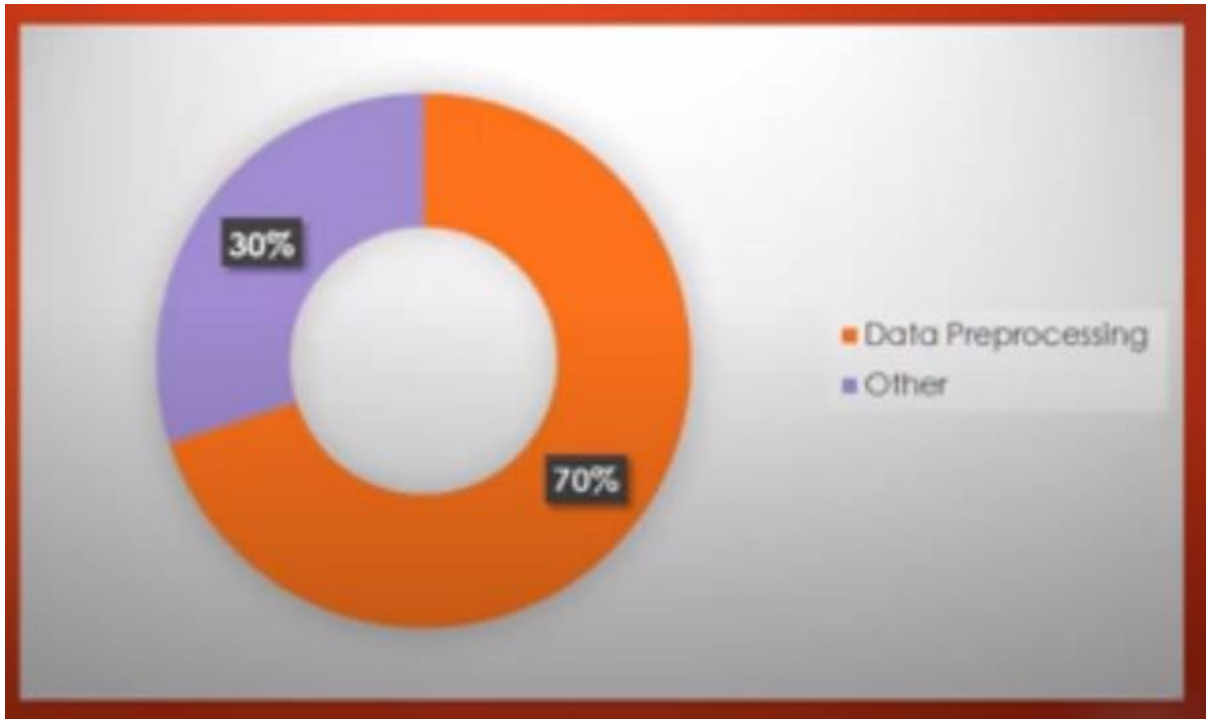
Data Preprocessing

Data Scientist most Spending Time!



Data Preprocessing

Data Preprocessing

To Convert raw data into valuable data,
we use Data Preprocessing.

Data Preprocessing

Why we use data preprocessing?

To perform better future prediction results, we have need to process data.

Data Preprocessing

Data Preprocessing Techniques

- ✓ Data Cleaning
- ✓ Data Integration
- ✓ Data Reduction
- ✓ Data Transformation

Data Preprocessing Techniques

1. Data Cleaning

Data cleaning is the process of fixing or removing incorrect, incomplete, corrupted, duplicate data within a dataset.

- ✓ Handling Missing Values (Nan Values)

Age	Income
32	60,000
33	Nan
36	90,000

32	60,000as
33	65,00as
36	90,000as

- ✓ Removing Strings
  - Removing Strings with Integer
  - Find and Remove Strings in the Dataset

- ✓ Removing Outlier
  - Min Outlier
  - Max Outlier

item	Price	
1	60	Min Outlier
2	650	Valuable Data
3	900	
4	950	
5	550	
6	56,890	Max Outlier

Data Preprocessing Techniques

2. Data Integration

Combining the data from different source
is called Data Integration.

Google Analytics

Join

Data Integration

Data Preprocessing Techniques

2. Data Integration

Combining the data from different source is called Data Integration.

Age	Class
12	4
15	8

City	Area
Lahore	Gulberb
Islamabad	Askari-10

After Integration

Age	Class	City	Area
12	4	Lahore	Gulberb
15	8	Islamabad	Askari-10

Data Preprocessing Techniques

3. Data Reduction

Reducing the data volume to fasten the model training.

Data Preprocessing Techniques

3. Data Reduction

Reducing the data volume to fasten the model training.

Mix	MN	Into	As
45	bdc	146	Gge5
33	bd33	14	55gh
789	x2d	365	c25c

Mix	Into
45	146
33	14
789	365

Data Reduction Techniques

- ✓ Dimension Reduction
- ✓ Numerosity Reduction

Data Preprocessing Techniques

4. Data Transformation

Data Transformation is the last stage of Data Preprocessing.
After this our data is ready to fit on the model.

Techniques of Data Transformation

Feature Engineering (Feature Scaling)

- ✓ Min Max Scaler
- ✓ Standard Scaler
- ✓ Max Abs Scaler
- ✓ Robust Scaler
- ✓ Quantile Transformer Scaler
- ✓ Log Transformation

etc....

DATA PREPOCESSING CODE IN PYTHON

```
import numpy as np
import pandas as pd
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import LabelEncoder

dataset = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/EmployData.csv')
```

create depedent and indepedent variable vector

```
x = dataset.iloc[:,:-1].values
print(x)
```

```
[['Mumbai' 3.0 51000.0]
 ['Newyork' 27.0 48000.0]
 ['Mumbai' 30.0 52000.0]
 ['Newyork' nan 66000.0]
 ['Tokyo' 48.0 nan]
 ['Tokyo' nan 51000.0]
 ['Singapore' 33.0 69000.0]
 ['Newyork' 105.0 79000.0]
 ['Newyork' 35.0 56000.0]
 ['Tokyo' nan 50000.0]
 ['Newyork' 105.0 79000.0]
 ['Mumbai' 2.0 76543.0]]
```

```
y = dataset.iloc[:,-1].values

print(y)
```

```
['Yes' 'Yes' 'No' 'No' 'Yes' 'No' 'No' 'Yes' 'No' 'No' 'Yes' 'Yes']
```

handle missing values

```
print(dataset.isnull().sum())
```

```
City                 0
Age                  3
Salary               1
Eligible for bonus    0
dtype: int64
```

```
dataset.dropna(inplace=True)
print(dataset)
```

```
         City    Age   Salary Eligible for bonus
0      Mumbai    3.0  51000.0                Yes
1     Newyork   27.0  48000.0                Yes
2      Mumbai   30.0  52000.0                 No
6   Singapore   33.0  69000.0                 No
7     Newyork  105.0  79000.0                Yes
8     Newyork   35.0  56000.0                 No
10    Newyork  105.0  79000.0                Yes
11     Mumbai    2.0  76543.0                Yes
```

```
print(dataset.describe())
```

```
                Age         Salary
count     9.000000      11.000000
mean     43.111111   61594.818182
std      38.050770   12532.728967
min       2.000000   48000.000000
25%      27.000000   51000.000000
50%      33.000000   56000.000000
75%      48.000000   72771.500000
max     105.000000   79000.000000
```

```
from sklearn.impute import SimpleImputer
```

```
imputer = SimpleImputer(missing_values=np.nan,strategy='mean')
#imputer = SimpleImputer(missing_values=np.nan,strategy='median')
#imputer = SimpleImputer(missing_values=np.nan,strategy='mode')
#imputer = SimpleImputer(missing_values=np.nan,strategy='constant',fill_value=10000)
```

```
imputer.fit(x[:,1:3])
```

```
▾ SimpleImputer
SimpleImputer()
```

```
x[:,1:3]=imputer.transform(x[:,1:3])
```

```
print(dataset.describe())
print(x[:,1:3])
```

```
                Age         Salary
count     9.000000      11.000000
mean     43.111111   61594.818182
std      38.050770   12532.728967
min       2.000000   48000.000000
25%      27.000000   51000.000000
50%      33.000000   56000.000000
75%      48.000000   72771.500000
max     105.000000   79000.000000
[[3.0 51000.0]
 [27.0 48000.0]
 [30.0 52000.0]
 [43.111111111111114 66000.0]
 [48.0 61594.818181818184]
 [43.111111111111114 51000.0]
 [33.0 69000.0]
 [105.0 79000.0]
 [35.0 56000.0]
 [43.111111111111114 50000.0]
 [105.0 79000.0]
 [2.0 76543.0]]
```

One Hot Encoding

```
from sklearn.compose import ColumnTransformer
```

```
from sklearn.preprocessing import OneHotEncoder
```

```
ct = ColumnTransformer(transformers=[('encoder',OneHotEncoder(),[0])],remainder="passthrough")
```

```
print(ct)
```

```
ColumnTransformer(remainder='passthrough',
                  transformers=[('encoder', OneHotEncoder(), [0])])
```

```
x = np.array(ct.fit_transform(x))
```

```
print (x)
```

```
[[1.0 0.0 0.0 0.0 3.0 51000.0]
 [0.0 1.0 0.0 0.0 27.0 48000.0]
 [1.0 0.0 0.0 0.0 30.0 52000.0]
 [0.0 1.0 0.0 0.0 43.111111111111114 66000.0]
 [0.0 0.0 0.0 1.0 48.0 61594.818181818184]
 [0.0 0.0 0.0 1.0 43.111111111111114 51000.0]
 [0.0 0.0 1.0 0.0 33.0 69000.0]
 [0.0 1.0 0.0 0.0 105.0 79000.0]
 [0.0 1.0 0.0 0.0 35.0 56000.0]
 [0.0 0.0 0.0 1.0 43.111111111111114 50000.0]
 [0.0 1.0 0.0 0.0 105.0 79000.0]
 [1.0 0.0 0.0 0.0 2.0 76543.0]]
```

Label Encoding

```
from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()
```

```
y= le.fit_transform(y)
```

```
print(y)
```

```
[1 1 0 0 1 0 0 1 0 0 1 1]
```

split the dataset for training and testing

```
from sklearn.model_selection import train_test_split
```

```
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.25,random_state = 1)
```

```
print(x_train)
```

```
[[0.0 1.0 0.0 0.0 105.0 79000.0]
 [0.0 1.0 0.0 0.0 27.0 48000.0]
 [0.0 0.0 1.0 0.0 33.0 69000.0]
 [1.0 0.0 0.0 0.0 3.0 51000.0]
 [0.0 1.0 0.0 0.0 105.0 79000.0]
 [1.0 0.0 0.0 0.0 2.0 76543.0]
 [0.0 0.0 0.0 1.0 43.111111111111114 50000.0]
 [0.0 1.0 0.0 0.0 35.0 56000.0]
 [0.0 0.0 0.0 1.0 43.111111111111114 51000.0]]
```

```
print(x_test)
```

```
[[1.0 0.0 0.0 0.0 30.0 52000.0]
 [0.0 1.0 0.0 0.0 43.111111111111114 66000.0]
 [0.0 0.0 0.0 1.0 48.0 61594.818181818184]]
```

```
print(y_train)
```

```
[1 1 0 1 1 1 0 0 0]
```

```
print(y_test)
```

```
[0 0 1]
```